

swarm-control

the plan, for one last look

A small harness that lets two teammates run several AI coding agents in parallel across two laptops, with Notion as the shared board and GitHub as the truth, to build the selective-hearing project without one giant model session eating the token budget.

Agents execute autonomously in fresh, bounded contexts.

Humans own intent through Notion.

Merged, verified code is the only proof of progress.

What you get

One command on each laptop.
Tasks flow from a plan to Notion
to the best agent to a reviewed
PR to main, with questions
surfacing in Notion when a human
is needed.

Why it saves tokens

Every task is a fresh, size-capped
session. Cheap work goes to
cheap models and other
providers. Fable and GPT-6 Astra
are reserved for critical tasks.

What it is not

Not a framework, not a self-
replanning AI company, not the
hackathon project. About 3,000
lines of Python plus tests, built
and rehearsed before Oct 10.

Prepared

September 26, 2026

For

Aryan and teammate, HackRU Fall 2026

Status

Design complete. Waiting for green light before writing the build plan.

Source

`swarm-control/docs/superpowers/specs/2026-09-26-swarm-control-design.md`
(full spec, 20 sections)

CONTENTS

01	The problem this solves	10	Budget and subscription limits
02	The system at a glance	11	Agent CLIs and models (verified)
03	What lives in Notion	12	Failure handling
04	Life of a task	13	Rules, security, and disclosure
05	Routing: best agent, right model	14	Testing and rehearsal
06	The worker loop	15	Game day, hour by hour
07	The control loop: review, merge, recover	16	Build phases and what I need from you
08	The human channel	17	Risks and open items
09	Shared docs and what each agent sees	18	Research this is based on

How to read this

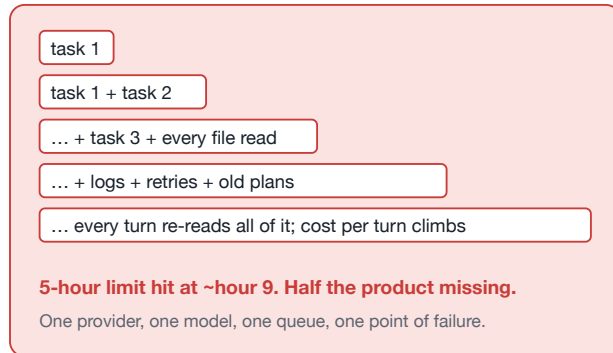
Each section is one idea with one diagram. The diagrams show the real mechanism, not decoration. If a section looks wrong, say the number and what to change. Everything here is also in the written spec, which is what the build plan will be generated from.

What I need you to decide after reading: (1) green light on the design as a whole; (2) whether the default agent roster and strength scores in section 05 look right for the two of you; (3) whether laptop A (yours) is the one that runs the control loop and the orchestrator session.

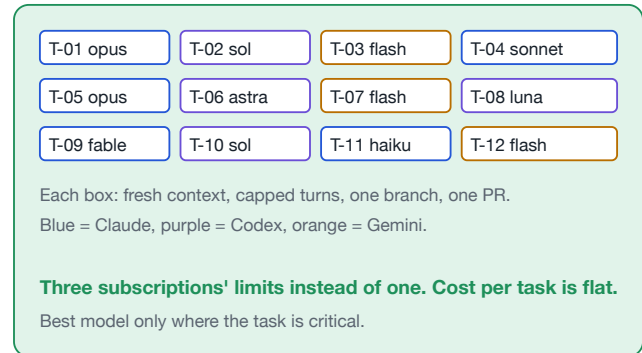
The problem this solves

One long Fable session accumulates context, gets slower and more expensive every turn, and hits the 5-hour limit before the product is done. The swarm replaces one long session with many short, bounded ones, spread across providers.

One session, all day



Many bounded sessions, in parallel



Left: what happens with a single long session. Right: the same work as bounded tasks spread over providers and model tiers.

What you asked for

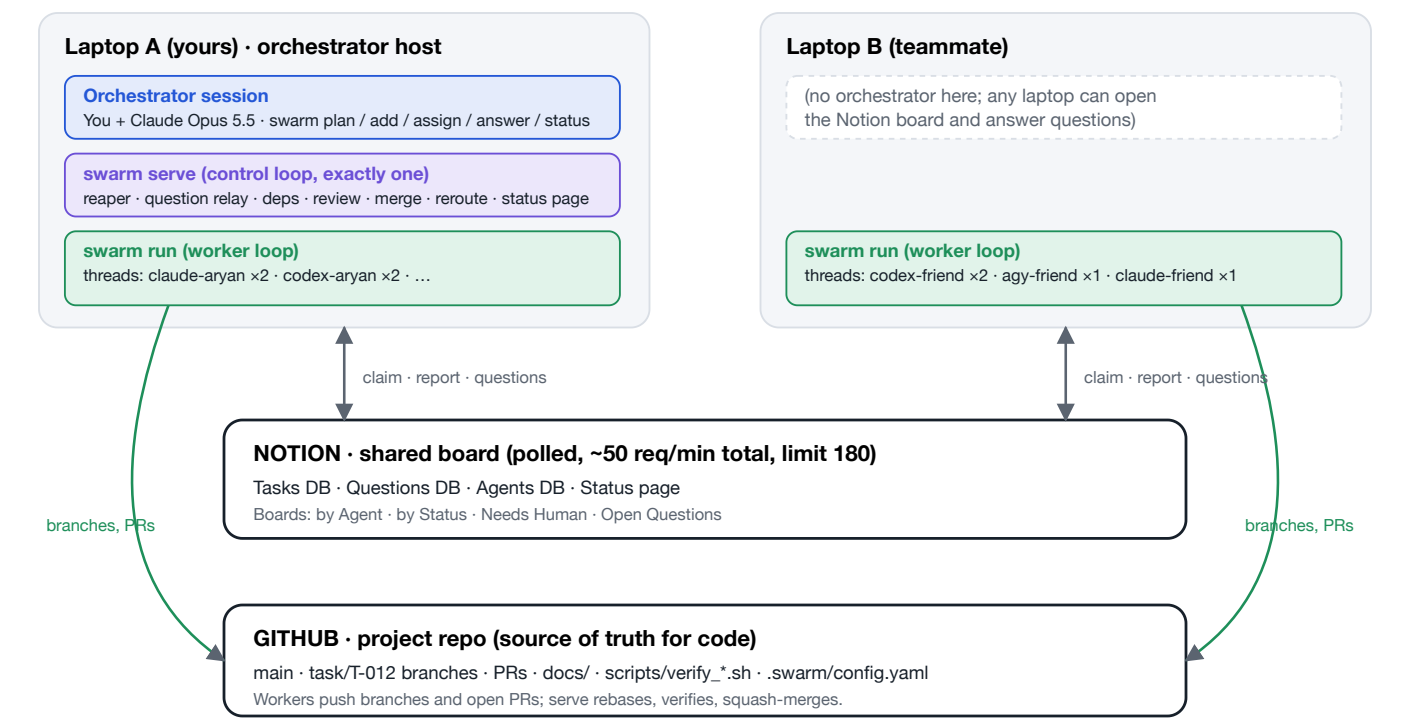
One orchestrator that splits a plan into tasks in Notion. Each AI takes tasks from its column, works on a branch, reports back, and the orchestrator merges. A reviewer watching everything. A Notion view where agents ask you questions. Shared docs so every agent knows the whole app. The best AI and model for each kind of task.

What I changed and why

Columns are the *output* of routing, not fixed lanes, so the best agent gets each task. A small Python tool drives the loop deterministically instead of trusting a model to remember to loop. Merges are automatic when checks pass, manual in the last hours. Your friend's plan supplied worktrees, typed reports, debt tracking, and evidence over claims; I dropped the parts two people can't run in a day.

The system at a glance

Three processes, two shared services. Notion is where humans look and steer. GitHub is where code lives. Each laptop runs one worker process for all its agents; one laptop also runs the control loop and the orchestrator session.

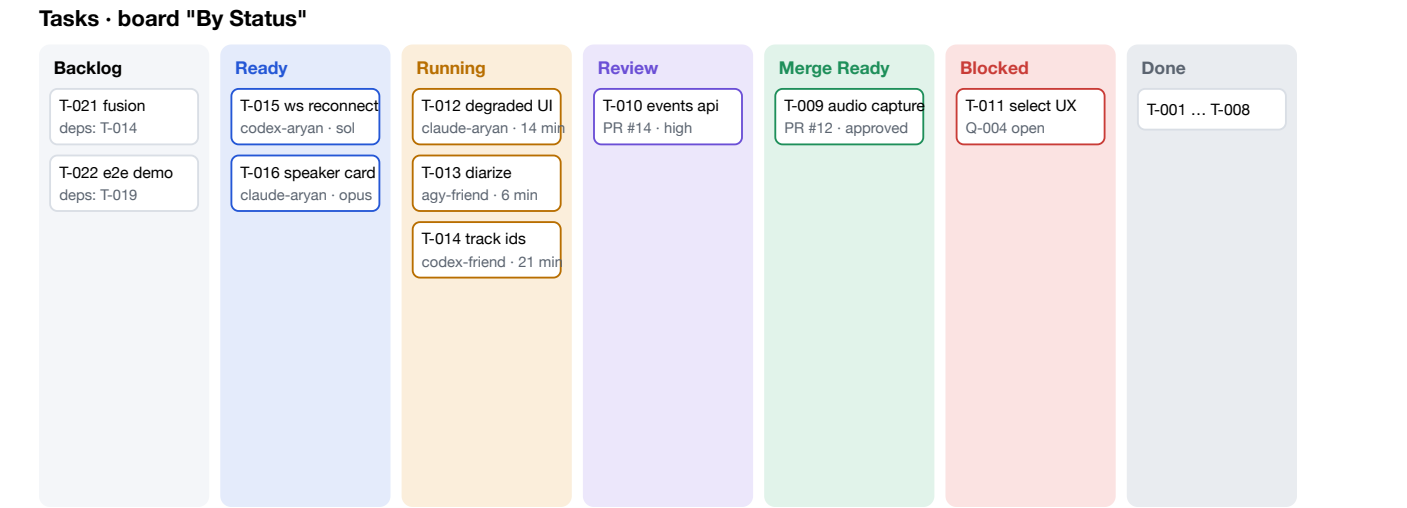


Every arrow is a plain HTTPS or git call. No webhooks, no tunnels, no server to host. If Notion is down, workers finish locally and publish later; git state is never corrupted.

Process	Where	Job
swarm run	one per laptop	Polls the board once per cycle for all agents on this host. Each agent thread: claim a task, make a worktree, build a prompt, run the CLI headless, verify, push, open a PR, report.
swarm serve	laptop A only	Recovers dead tasks, relays your answers to blocked tasks, promotes tasks whose dependencies merged, runs the reviewer, merges, reroutes around rate limits, rewrites the status page every 15 minutes.
orchestrator session	laptop A	Interactive Claude Code in the project repo. Turns PLAN.md into tasks a milestone at a time, reads the status page, re-plans, answers questions. Never reads code.

What lives in Notion

Three databases and one page, created by `swarm init` with board views already configured. Humans only ever type in two fields: a question's Answer, and its Needs Follow-up checkbox. Everything else is written by the tool, and dragging a card between agent columns is respected.



Also: Changes Requested (back to the same worker), Failed (retry ladder), Cut. A second board groups the same cards by Agent.

The board you'll actually look at. Each card carries type, importance, size, agent, model, branch, PR, attempts, and its reports in the page body.

Tasks

ID, title, description, acceptance criteria, type, importance, size, milestone, priority, agent, model, effort, status, depends-on, scope (globs the worker may edit), feedback, branch, PR, attempts, timestamps. Reports append to the page body per attempt.

Questions

Question, kind (blocking or fyi), context, options, "proceeding with", impact, linked task, asked by, status, **Answer**, **Needs Follow-up**. The Open column is your inbox.

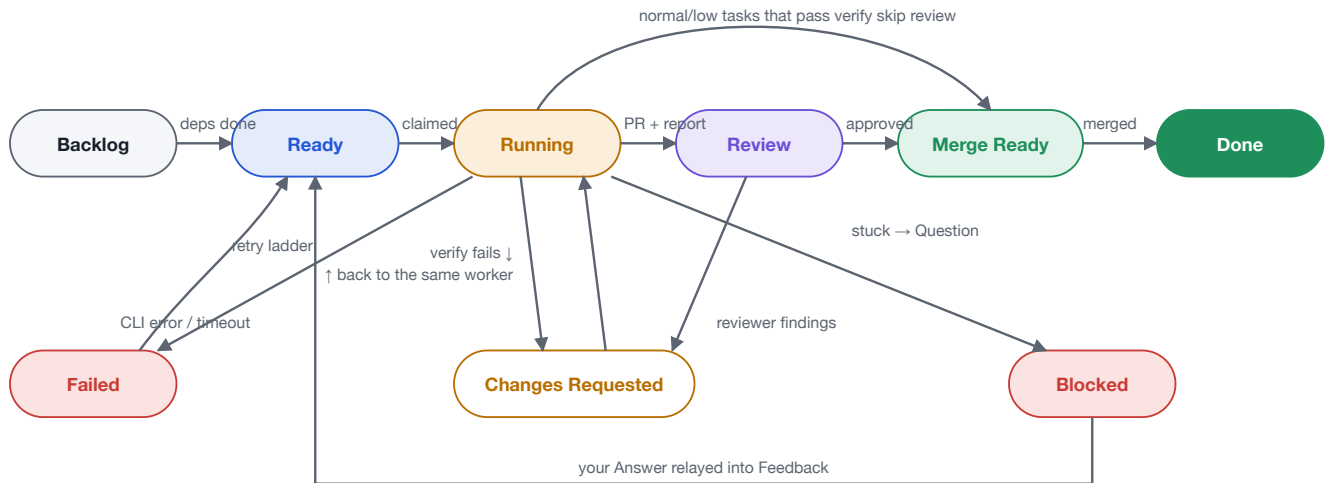
Agents

Name, provider, laptop, status (idle, running, cooldown, offline), current task, last heartbeat, cooldown-until, runs, tokens in and out, cost. Both laptops see each other's health here.

Notion has no atomic "claim". Contention is avoided by design because each worker only claims cards in its own agent column, and guarded by writing a random nonce, waiting 1.5 s, and re-reading. All writes retry on rate limits honoring Retry-After.

Life of a task

A task moves through a fixed set of states. Every transition is made by the tool, from evidence, never from an agent's claim that it is done.



Retry ladder for Failed: attempt 1 same agent · attempt 2 one model tier up (other agent if close) · attempt 3 asks you.
 Stale Running (no heartbeat 10 min) → Ready. A rebase conflict at merge also sends the task to Changes Requested.

State machine for one task. Solid arrows are transitions the tool makes from verify results, reviewer verdicts, merge outcomes, or your answers.

What "done" means

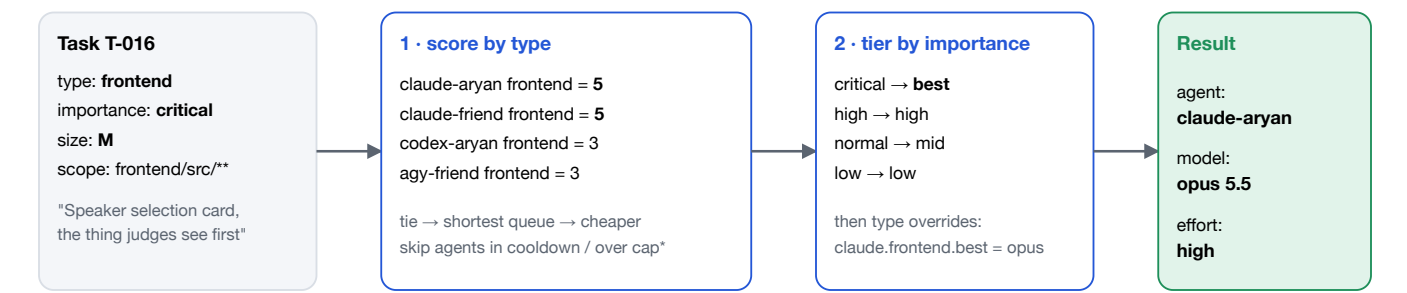
The branch was rebased on the latest main, the fast verify script passed on the rebased tree, the PR was squash-merged, and the task's decisions and temporary hacks were appended to the repo's decision and debt logs. Not "the agent said it finished."

When review is skipped

Default policy reviews critical and high tasks, anything that touched the contracts or design docs, anything with a missing report, and anything that edited files outside its scope. Normal and low tasks that pass verify merge directly. In the last four hours you flip the policy to review everything and merge by hand.

Routing: best agent, right model

Every task carries a type, an importance, and a size. A deterministic table lookup picks the agent with the highest strength for that type, then the model tier from importance. No model decides who works on what.



* critical tasks ignore cooldown and cap filters; they wait for the best agent unless another scores within one point. Scope overlap prefers the agent already working in those files.

Your example, traced. A critical frontend task lands on Claude running Opus 5.5, because Opus currently outranks Fable on the WebDev Arena leaderboard. Fable is used for critical, large, long-horizon tasks where it leads.

Default strength table (1 = weak, 5 = strongest), editable in config

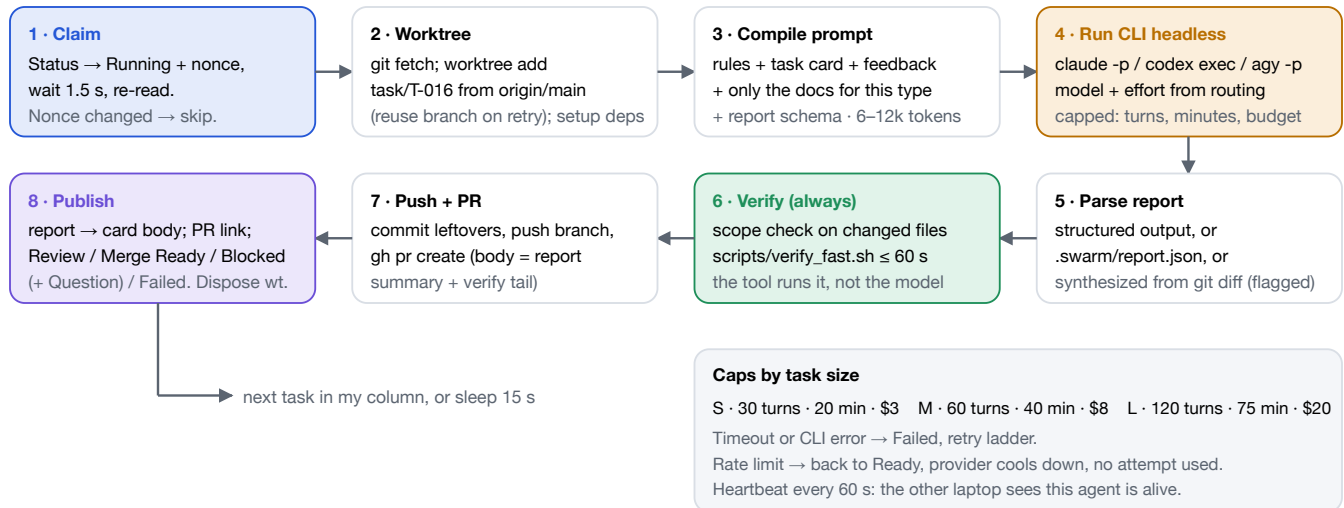
Task type	Claude Code	Codex	Gemini / Antigravity	Evidence behind the default
frontend, docs, research, bugfix, integration	5	3–4	3–4	WebDev Arena (Sep 25 2026): Opus 5.5 #1, Astra #2, Fable #3. Practitioner reviews: Claude for design and multi-file work.
backend, realtime, tests, infra	4	5	2–3	Reviews agree Codex "does what you said" on backend; strong at PR-review bug catching.
ml_audio, ml_fusion, eval	4	3–4	3–4	Terminal-Bench 2.1: Fable 5.1 tops; MLE-bench: Gemini 3 Pro and Opus within 1.3 points.
ml_vision (video/image input)	4	3	4	Gemini accepts video and audio natively; cheapest bulk option.

Tier	Claude	Codex	Antigravity	Used for
best	Fable 5.1 (Opus 5.5 for frontend)	GPT-6 Astra · xhigh	Gemini 3.1 Pro	critical only
high	Opus 5.5 · high	GPT-6 Astra · high	Gemini 3.1 Pro	high
mid	Sonnet 5 · medium	GPT-6 Sol · medium	Gemini 3.8 Flash	normal
low	Haiku 4.5	GPT-6 Luna	Gemini 3.8 Flash	low

Override any assignment with `swarm assign T-016 --agent codex-aryan --model gpt-6-astra`, or drag the card to another agent's column. Rerouting runs automatically for Ready tasks whose agent is offline, rate-limited, or over its spend cap.

The worker loop

One `swarm run` per laptop. It polls once for all its agents, hands each task to a free slot, and runs the eight steps below in an isolated git worktree. The model only edits and commits; the tool pushes, opens the PR, and reports.



The report the model must produce: status, summary, files changed, tests run, temporary hacks (debts), decisions with impact, and an optional question. Claude and Codex return it as validated structured output; other CLIs write a file.

Why a tool drives the loop

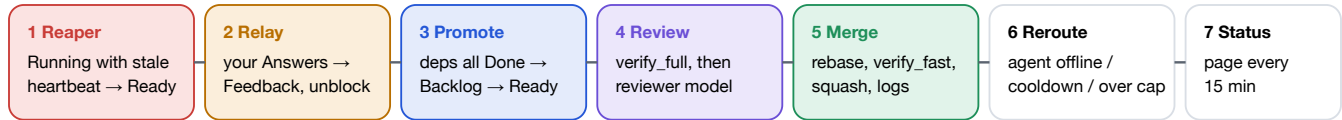
A model asked to "keep looping" forgets, drifts, or stops. A 300-line Python loop does not. It also gives you restarts: kill it, start it again, and it resumes from the board.

Why the model never pushes

Codex stays in its write-only sandbox, Claude needs no network permission, and nothing an agent does can reach GitHub except through the tool with your own `gh` login.

The control loop: review, merge, recover

`swarm serve` runs on laptop A every 30 seconds and does the seven things below, in order. It is the "agent that constantly reviews all the work" you asked for, split into a deterministic part (verify, rebase, merge, recover) and a model part (the reviewer).



Reviewer (model part)

Gate: does policy require review? (critical/high, contracts or design docs touched, missing report, out-of-scope edits). Else → Merge Ready.

Deterministic first: fresh worktree, `verify_full.sh`.

Fails → Changes Requested, no model tokens spent.

Then a **different model family** (Codex GPT-6 Sol by default), read-only, gets acceptance criteria + diff + worker report + verify output.

Verdict: approve · request changes (≤ 2 rounds) · escalate → Question.

Status page (the "are we on track" view)

13.5 h left · claude ok · codex ok · agy cooldown 9 m

M1 vertical slice 8/8 done

M2 core 11/17 done · 3 running · 1 blocked

M3 polish 0/9 · not started

Needs you: Q-004 (T-011 select UX), T-018 failed ×3

Spend 5h: claude \$31/40 · codex 1.2M tok · agy 0.4M tok

RISK: M2 has 6 open tasks and 3.5 h before M3 must start.

Whether to re-plan stays a human call. You read the risk line, then add, cut, split, or reassign tasks from the orchestrator session in one command each.

Merge, exactly

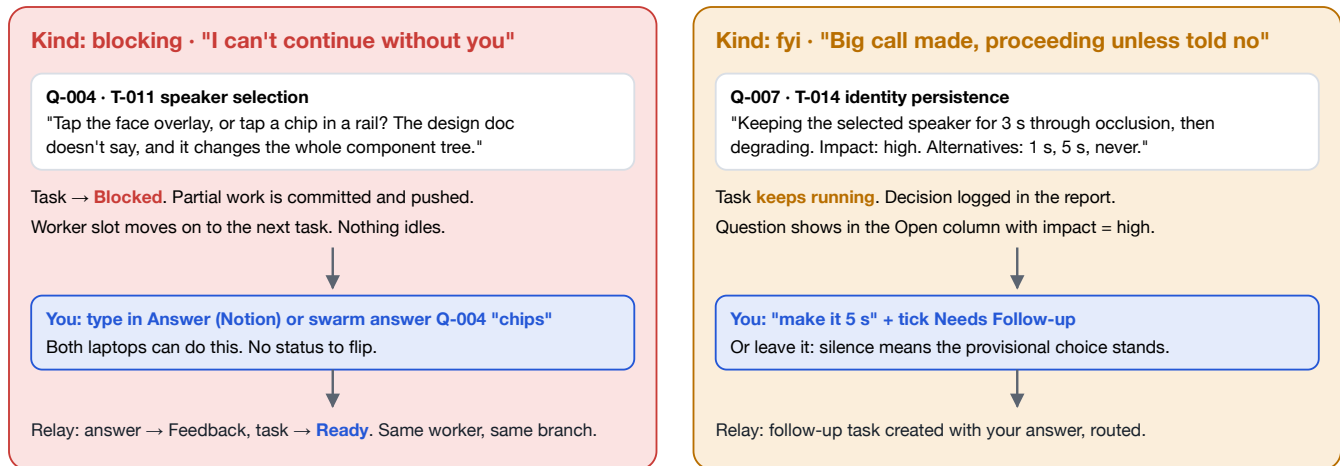
Fresh worktree of the branch, rebase onto main, run fast verify on the rebased tree, force-push with lease, squash-merge via `gh`, mark Done, append the task's decisions and debts to `docs/DECISIONS.md` and `docs/DEBT.md`, then promote dependents. Conflicts go back to the same worker with the file list.

Two modes

Automatic merging is the default because a broken main is caught by the verify script before it lands. At T-4h you switch to `--manual`: `serve` lists Merge Ready PRs and you pick which land. Only one `serve` can run; a lock row in the Agents DB stops a second one.

The human channel

Agents never stop for reversible decisions. They ask in Notion in two ways, and you answer in Notion or in the orchestrator session. Both write the same field, and the control loop carries the answer to the right task within 30 seconds.



Two kinds of question, two outcomes. The one lesson from the only published attempt at "Notion as a Claude Code queue" was that it died for lack of exactly this channel.

Who asks

Workers (stuck, or a high-impact call), the reviewer (escalate after two rounds), and the planner (ambiguity in PLAN.md).

Where it shows

Questions board, Open column, sorted by impact. Also the "Needs you" line on the status page and in `swarm status`.

Paper trail

Every decision agents report is appended to `docs/DECISIONS.md` on merge, so the pitch and judge Q&A have receipts.

Shared docs and what each agent sees

Every agent on every laptop reads the same docs because they live in the repo and each worktree is cut from the latest main. Nobody gets everything. The prompt inlines only the docs for the task's type and points to the rest by path.

File	Holds	Who writes it
PLAN.md	Product, demo story, milestones, success criteria. The input to <code>swarm plan</code> .	You, with the orchestrator, at kickoff; orchestrator thereafter
docs/ARCHITECTURE.md	Components, data flow, how to run, the ML pipeline section.	Backend and integration tasks propose; orchestrator approves
docs/DESIGN.md	UX principles, tokens, the UI state machine, component list, screenshots.	Frontend tasks (always reviewed)
docs/CONTRACTS.md	API endpoints, WebSocket events, JSON schemas.	Backend tasks (always reviewed; must list affected consumers)
docs/DECISIONS.md , docs/DEBT.md	Append-only logs of decisions and temporary hacks.	The merge step
AGENTS.md (+ one-line CLAUDE.md , GEMINI.md)	Worker rules: stay in scope, commit as you go, run verify, list every hack, ask via a question, end with the report.	Ships with the template

Docs inlined into the prompt, by task type					
	PLAN.md#summary	AGENTS.md	DESIGN.md	CONTRACTS.md	ARCHITECTURE.md
frontend	☒	☒	☒	☒	☐
backend / realtime	☒	☒	☐	☒	☒
ml_audio / vision / fusion	☒	☒	☐	☒ #events	☒ #ml
integration / docs	☒	☒	☒	☒	☒
tests / eval / bugfix	☒	☒	☐	☐	☐

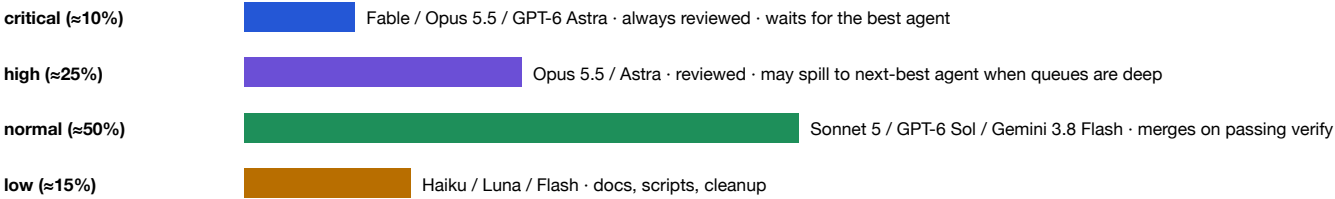
Filled = inlined. Hollow = referenced by path; the agent reads it on demand. A "#section" pulls one heading only. Target prompt size is 6–12k tokens including the task card and report schema.

Your ask, answered: yes to a plan doc and a design doc. I added contracts (so frontend, backend, and ML agree on event shapes without talking) and architecture (so integration tasks know how pieces fit). The mapping above is config, so you can change who sees what on the day.

Budget and subscription limits

This is the reason the system exists. Seven mechanisms, all configuration, keep any one subscription from becoming the bottleneck and keep the expensive models for the work that deserves them.

Where tokens go, by importance (illustrative share of ~40 tasks)



API list price per 1M output tokens, for scale: Fable \$50 · Astra \$50 · Opus 5.5 \$20 · Sonnet 5 \$10 · Sol \$10 · Flash \$3.75 · Haiku \$5 · Luna \$0.50.
Your subscriptions cover the runs; these numbers only show why routing normal work away from the top tier matters.

Mechanism	What it does	Default
Fresh context per task	No session outlives one task. Cost per task is flat instead of climbing.	S 30 turns · M 60 · L 120
Tier by importance	Best models only for critical work; evidence-based exceptions per type.	see section 05
Concurrency caps	Per provider per laptop, because Claude subscriptions have been observed to 429 around six parallel sessions.	claude 3 · codex 3 · agy 2
Usage ledger	Every run logs tokens and cost (where the CLI reports it) per agent, to a local file and the Agents DB.	always on
Soft caps	Above 80% of an agent's 5-hour cap, it stops receiving normal and low work.	claude \$40 / 5 h, tunable
Cooldown + reroute	A rate-limit error parks the provider until its reset time (or 15 min) and reroutes non-critical Ready tasks. No attempt is consumed.	automatic
Cheap orchestration	Orchestrator on Opus 5.5, never reads code. Planner is one structured call per milestone. Reviewer on GPT-6 Sol, only on high+ tasks.	config

Fable specifically: on your Max plan Fable is included only up to half the weekly limit, and it is not the top model for frontend. The default config spends Fable only on critical, large, long-horizon tasks (multi-file ML and integration), which is where the benchmarks put it first.

Agent CLIs and models (verified Sep 26, 2026)

One adapter per provider behind one interface. The roster you bring on the day is config; a generic adapter covers anything with a prompt flag (Copilot CLI, Cursor CLI, Muse, or whatever appears by October).

Provider	Headless invocation the runner uses	Status
Claude Code v2.1.283	<code>claude -p --output-format json --permission-mode bypassPermissions --model {m} --effort {e} --max-turns {n} --max-budget-usd {b} --append-system-prompt-file rules.md --json-schema {report}</code> with the prompt on stdin. Returns cost, usage, session id, and validated structured output.	verified
Codex v0.157.1	<code>codex exec --json -C {wt} -m {m} -c model_reasoning_effort={e} --sandbox workspace-write --output-schema report.json -.</code> JSONL events with token usage (no dollar figure).	verified
Antigravity replaces Gemini CLI	<code>agy -p ... --model={m} --approval-mode yolo --output-format json</code> . Gemini CLI dropped personal and AI Pro accounts on Jun 18, 2026; your Gemini Pro subscription works in Antigravity. Flags are third-party sourced.	experimental, verify at rehearsal
Gemini CLI API key only	<code>gemini -p --output-format json --approval-mode yolo -m {m}</code> . Fallback if Antigravity fails; free API key is Flash-only, 250 req/day.	verified
Grok Build v1.0.41	<code>grok -p --prompt-file p.md --output-format json --always-approve --max-turns {n} --cwd {wt} --model {m}</code> . Needs SuperGrok or an xAI key.	verified
generic	A command template with <code>{prompt_file}</code> <code>{model}</code> <code>{cwd}</code> placeholders; report via file only.	any

Models and knobs

Family	Selectable	Reasoning knob	Subscription notes
Anthropic	fable (5.1), opus (5.5, default), sonnet (5), haiku (4.5); 1M context on Opus, Sonnet, Fable	<code>--effort low...max</code>	Pro: Opus/Sonnet/Haiku, Fable by credits. Max: Fable up to 50% of weekly. 5-hour rolling window shared with claude.ai.
OpenAI	gpt-6-astra (best), gpt-6-sol (mid), gpt-6-luna (cheap); gpt-5.5 retires Oct 14	<code>model_reasoning_effort low...ultra</code>	Plus per 5 h: Astra 5–45, Sol 15–150, Luna 350–3,000 messages. Pro 5×/20× multiply that. CLI and cloud share one pool.
Google	gemini-3.1-pro, gemini-3.8-flash (Antigravity)	none documented	AI Pro: 5-hour refresh plus weekly cap, no published numbers. Free: "meaningful weekly quota."

Failure handling

Everything that can fail on a hackathon night, and what the tool does about it without waking you up.

Failure	Detected by	Response
Worker process dies or laptop sleeps	heartbeat older than 10 min	Task back to Ready, attempts +1, pushed branch reused. At most one task's uncommitted work is lost.
CLI error or timeout	exit code, timer	Failed → retry ladder: same agent, then one tier up (other agent if within one point), then a question to you.
Rate limit on a provider	parsed error	Provider in cooldown until reset (or 15 min), visible on both laptops. Task back to Ready, no attempt spent. Non-critical work rerouted.
Verify script fails	non-zero exit	Changes Requested with the failure tail; same worker, same branch.
Rebase conflict at merge	git	Changes Requested with the conflicting file list; same worker resolves.
Notion unreachable	HTTP errors after 5 retries	Worker finishes locally and retries publishing; serve skips the cycle. Git state is never touched by Notion state.
Provider CLI missing on a laptop	swarm doctor , spawn error	Agent marked offline; its Ready tasks rerouted.
Model didn't write the report	parser	Report synthesized from the diff and verify output, flagged, forced through review regardless of policy.
Model edited files outside its scope	diff vs scope globs	Flagged in the report; forced through review. Not auto-reverted, because that could destroy the actual work.
Two control loops started	lock row in Agents DB	Second refuses to start unless the first's heartbeat is stale.
Task type nobody is strong at	router	Assigned to the top scorer anyway with a note. Nothing is ever left unassigned.
Antigravity adapter broken	doctor smoke test	Agent disabled; Gemini via API key or the other providers take its share.

The recovery drill in rehearsal: start ten tasks over three agents on two laptops, kill one `swarm run` mid-task, and confirm the task is reaped and finished by the next attempt, that nothing merges twice, and that the status page reflects it within a minute.

Rules, security, and disclosure

HackRU and MLH rules (verified)

- **Oct 10–11, 2026**, Busch Student Center, 24 hours, hacking 12:00 PM Saturday to 12:00 PM Sunday, teams of up to 4.
- Fall 2025 rules (2026 not yet posted): "You are not allowed to submit a project that has been built before the start of hacking." MLH standard rules add that teams may use libraries, frameworks, and open-source code, and must allow generative AI tools.
- MLH asks teams to be transparent about AI tools used, listing them in the submission.
- Judging: functionality and execution, creativity and innovation, technical difficulty. In-person table judging, no demo video required, one track only.

How we stay clean: swarm-control is a dev tool in its own public repo, dated before Oct 10, never submitted as the project. The project template contains only structure, empty docs, and verify scripts, no product code. The Devpost writeup lists the harness and every model used. I'd still email the organizers a one-liner to confirm; it costs nothing.

Security and blast radius

- Worker CLIs run with full tool permission, but only inside a worktree of the project repo. Never run `swarm run` from a home directory.
- Secrets are environment variables. `.env` is gitignored. Prompts never include them.
- Codex keeps its workspace-write sandbox. Claude's bypass mode is accepted for a hackathon.
- Only the tool pushes and merges, using your own `gh` login. Models never touch the network.
- Nothing deletes branches except the squash-merge cleanup after success.
- Notion: one internal connection token shared by both laptops, scoped to one parent page. Teammate joins as a page guest so a free workspace's 1,000-block cap for multi-member workspaces doesn't apply.

Testing and rehearsal

Four layers, from no-network unit tests to a real two-laptop rehearsal. The harness is "ready" only when the last layer has been run once, end to end, in under 90 minutes from kickoff to first merged PR.

1 · Unit (pytest)

config validation
router, every rule
prompt compiler + budget
report parser + fallbacks
adapters vs recorded output
Notion client, mocked

2 · End to end, local

in-memory board
bare git remote in tmp
fake agent script
6 tasks, 2 agents, deps
one blocked, one failed,
one reaped

3 · Notion integration

init on a throwaway page
create / claim / report
board views exist
forced 429 retry path
markdown report append

4 · Rehearsal

both laptops, real CLIs
toy project from template
10 tasks over 3 agents
kill a runner mid-task
force a review round
measure tokens per size

What rehearsal tunes

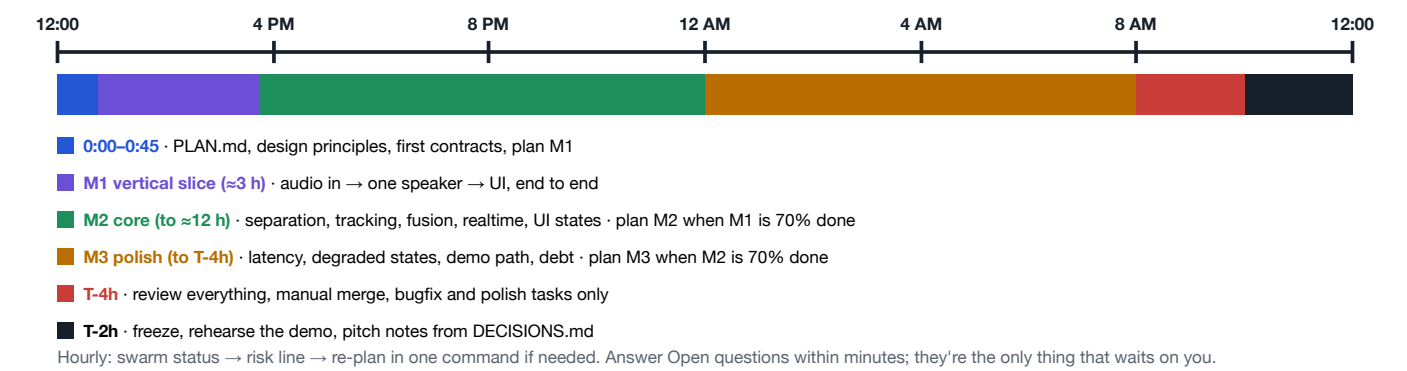
Turn and minute caps per task size, the Claude concurrency ceiling on your plan, soft caps, and whether the Antigravity adapter works headless. These are the numbers I can't know without running it.

What I'll report after it

A short table: tasks run, tokens per size, time per size, failures and how they recovered, and the config I changed. If something doesn't hold up, you'll hear it before Oct 10, not during.

Game day, hour by hour

Full runbook ships with the tool. This is the shape of the day.



When	Who	What
Before	both	swarm doctor green on both laptops. Notion page shared. Template repo instantiated. gh auth done. Laptop A on power with sleep disabled.
0:00–0:30	you two + orchestrator	Write PLAN.md : product, demo story, milestones. Sketch design principles and the first event contracts. This is the highest-leverage half hour of the day.
0:30–0:45	orchestrator	swarm plan PLAN.md --milestone M1 . Read the proposed tasks, fix any, apply.
0:45	laptop A, then both	caffeinate -i swarm serve ; swarm run on both laptops.
0:45–1:30	you two	Do not hand-code. Watch the Questions board, answer fast, fix the verify script if it misbehaves on the real project.
hourly	orchestrator	swarm status . Plan the next milestone at 70%. Cut or split anything the risk line names.
T-4h	you two	Flip to review-everything and manual merge. Only bugfix and polish tasks.
T-2h	you two	Freeze. Demo rehearsal. Presentation notes from the decisions log.

Build phases and what I need from you

Reliability first, intelligence last. Each phase leaves a working, tested tool.

#	Phase	Delivers	Proof it works
1	Models, config, board interface	Dataclasses, config loader and validation, Board protocol, in-memory board.	unit tests
2	Notion board	<code>init</code> , <code>create/claim/query/update</code> , markdown reports, views, retries.	integration test on a throwaway page
3	Workspace	Worktree <code>add/reuse/dispose</code> , <code>commit</code> , <code>push</code> , PR create via <code>gh</code> .	unit tests on a tmp bare remote
4	Adapters	Claude, Codex, generic first; Antigravity, Gemini, Grok after. Output parsing, rate-limit detection.	tests against recorded outputs; <code>doctor smoke</code>
5	Router + prompt + report	Routing rules, doc selection, report schema and fallbacks.	unit tests, <code>--dry-run</code>
6	Runner	<code>swarm run</code> : threads, claim, the eight steps, publish outcomes, heartbeat.	end-to-end with fake agent
7	Serve	Reaper, relay, promotion, merge, reroute, status page, lock.	end-to-end: blocked, failed, reaped
8	Reviewer + planner	Review gate and run; <code>swarm plan</code> with schema and conflict lint; <code>split</code> .	real run on a toy repo
9	Template + runbook + docs	<code>hackathon-base</code> , <code>ORCHESTRATOR.md</code> , <code>RUNBOOK.md</code> , <code>README</code> .	fresh clone to first PR in 90 min
10	Rehearsal	Both laptops, real CLIs, kill-and-recover, tune caps.	report to you

From you, before I start

- Green light on this document.
- A Notion page to be the parent, and an internal connection token (I'll give exact steps; five minutes).
- A throwaway GitHub repo for testing, and `gh auth login` on this Mac.
- Whether laptop A is yours.

From your teammate, before rehearsal

- Which CLIs and subscriptions they have (Codex on ChatGPT Plus or Pro? Claude? Gemini Pro?).
- Their laptop OS. Python 3.11+ and `gh` installed.
- Guest access to the Notion page.

After the green light I write the implementation `PLAN.md` and stop. No code until you say start.

Risks and open items

Risk	Likelihood	What I'm doing about it
Antigravity headless flags don't work as documented by third parties	medium	Adapter marked experimental; <code>doctor</code> smoke test disables it; Gemini via API key and the other providers absorb its share.
Notion public API blocked on a free workspace	low-medium	<code>doctor</code> tests it first thing. Fallback is one month of Plus on one workspace. A GitHub-Issues board backend is a possible second implementation but out of scope unless this fails.
Claude concurrency ceiling lower than three per laptop	medium	Caps are config; rehearsal measures the real number; cooldown and reroute handle surprises on the day.
Task decomposition too coupled, agents collide	medium-high	Scope globs per task, dependencies, contracts-first tasks in M0/M1, conflict lint in <code>swarm plan</code> , scope affinity in routing, rebase conflicts routed back to the same worker. Still the biggest human-skill factor on the day.
Verify scripts flaky on the real project	medium	Template ships lint + unit + smoke; first 45 minutes of game day include fixing them; a flaky verify sends tasks to Changes Requested, not to main.
Model ids or CLI flags change before Oct 10	medium	All ids and commands in config; <code>doctor</code> checks each; one-line edits.
Reviewer too lenient or too strict	low	Bounded to two rounds, then escalates to you. Policy tiers keep it off trivial work.
You get pulled into hand-coding and stop answering questions	medium	The runbook's rule for the first hour. Questions are the only thing that waits on you; everything else keeps moving.

Decisions I made for you (say so if you disagree)

- Notion over GitHub Issues as the board, because you asked for it and its risks are cheap to mitigate.
- A Python tool drives the loop; models don't loop themselves.
- Auto-merge on passing verify until T-4h, then manual.
- Orchestrator on Opus 5.5; reviewer on Codex GPT-6 Sol; Fable reserved for critical large tasks.
- Four shared docs, not two: plan, design, contracts, architecture.
- The harness lives in its own public repo and is disclosed.

Research this is based on

Four research passes on Sep 26, 2026, against official docs, leaderboards, and primary sources. Nothing in the design rests on memory alone.

Patterns that every working system converged on

- **Worktree per task, branch named after the task, cleanup on done.** ApiariTools/swarm, claude-squad, vibe-kanban, Conductor, Codex app.
- **Leased claims with bounded retries; a reaper requeues stale work.** desplega-ai/agent-swarm, the only open-source project found with real multi-machine workers.
- **Durable progress files that every session reads first.** Anthropic's long-running-agent harness (Nov 2025) and its quickstart.
- **One task per fresh context, capped iterations, tests as backpressure.** The "Ralph" loop and Anthropic's plugin.
- **Separate the agent doing the work from the agent judging it.** Anthropic, Mar 2026: "a strong lever," and evaluators only pay off near the capability boundary.
- **Keep a blocked / needs-human status.** The one published Notion-to-Claude-Code pipeline was abandoned for lack of it.

Facts verified against primary sources

- Notion API version 2026-03-11: data sources, database and view creation, markdown page endpoints, 180 req/min, no compare-and-set, webhooks need a public URL.
- Claude Code 2.1.283 headless flags, structured output, effort levels, session resume, and the reported 429 behavior around six concurrent sessions.
- Codex 0.157.1 `exec` flags, JSONL output, sandbox modes, reasoning effort, per-5-hour message estimates by plan.
- Gemini CLI personal-account shutdown (Jun 18, 2026) and the move to Antigravity; Grok Build headless mode.
- WebDev Arena (Sep 25, 2026), Terminal-Bench 2.1, MLE-bench, and 2026 practitioner comparisons behind the routing defaults, with the caveat that benchmark boards disagree and the newest models are days old.
- HackRU Fall 2026 dates and format; Fall 2025 rules; MLH member-event rules on prior work and AI tools.

Your teammate's PLAN.md supplied the invariants this borrows: worktrees, typed reports, debt records for every shortcut, decision receipts, evidence over claims, and a planner that stays high-level. What it left out on purpose: leases as a service, a capacity controller, a steering compiler, a chronicler agent, and an event bus. Two people in 24 hours need a tool, not a company.